

バックプロパゲーション

勾配を全パラメータに届ける仕組み

rkskmt

前回の振り返り

- 変換を重ねると**曲がる境界**が作れる（ReLU の折り目）
- 学習に必要なのは、**全パラメータの勾配 $\partial L / \partial \theta$**
- 前は「誤差が逆向きに流れる」とだけ言って、種明かしを保留した

📌 今回の問い

パラメータが5個、100個、100万個 — **全部の勾配**をどうやって計算する？

正面から微分すると地獄

- 前回の2層ネット。 α_1 の勾配を**直接**書き下すと：

$$\frac{\partial L}{\partial \alpha_1} = \frac{\partial}{\partial \alpha_1} \left[-\log \sigma \left(w \cdot \text{ReLU}(\alpha_1 x_1 + \alpha_2 x_2 + b) + c \right) \right]$$

❗ クイズ

パラメータは5個。この調子で5本の式を全部展開する？ 100層・100万パラメータになったら？

- 実は、**展開しなくていい**
- 鍵は高校数学で習った**合成関数の微分（連鎖律）**

連鎖律：微分は掛け算でつなげる

- 関数が数珠つなぎなら、微分も数珠つなぎ：

$$y = f(g(x)) \quad \Rightarrow \quad \frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$$

- NN は「アフィン → ReLU → アフィン → シグモイド → 損失」という**数珠つなぎそのもの**

$$\frac{\partial L}{\partial \alpha_1} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial u} \cdot \frac{\partial u}{\partial h} \cdot \frac{\partial h}{\partial z} \cdot \frac{\partial z}{\partial \alpha_1}$$

- 各カケラは**どれも1行で書ける簡単な微分**
- カケラを**出力側から順に**掛けて貯めていけば、途中の計算を**使い回せる**

今日のネットワーク（前回と同じ5パラメータ）

$$x \xrightarrow{z=\alpha_1x_1+\alpha_2x_2+b} z \xrightarrow{\text{ReLU}} h \xrightarrow{u=wh+c} u \xrightarrow{\sigma} p \xrightarrow{-\log} L$$

これを **具体的な数値で1往復** する：

記号	値
入力 x_1, x_2	1.0, 0.5
正解 y	1 (Salmon)
α_1, α_2, b	0.3, -0.2, 0.1
w, c	0.5, -0.1

往路（Forward）：数値を流す

$$\begin{aligned} z &= 0.3 \times 1.0 + (-0.2) \times 0.5 + 0.1 = 0.30 \\ h &= \text{ReLU}(0.30) = 0.30 \\ u &= 0.5 \times 0.30 + (-0.1) = 0.05 \\ p &= \sigma(0.05) = 0.5125 \\ L &= -\log(0.5125) = 0.6685 \end{aligned}$$

- 予測は 0.5125 — 「ほぼコイン投げ」。正解は 1 なので、もっと上げたい
- 途中の値 (z, h, u, p) は捨てずに取っておく** — 復路で全部使う

復路（Backward）：出力側から勾配を貯める

出力層

$$\delta_{\text{out}} = \frac{\partial L}{\partial u} = p - y = 0.5125 - 1 = -0.4875$$

（シグモイド＋交差エントロピーの「打ち消し合い」 — 最小単位の回の道具）

$$\frac{\partial L}{\partial w} = \delta_{\text{out}} \cdot h = -0.4875 \times 0.30 = -0.1463 \qquad \frac{\partial L}{\partial c} = \delta_{\text{out}} = -0.4875$$

中間層へ「逆流」

$$\delta_{\text{mid}} = \delta_{\text{out}} \cdot w \cdot \text{ReLU}'(z) = -0.4875 \times 0.5 \times 1 = -0.2438$$

$$\frac{\partial L}{\partial \alpha_1} = \delta_{\text{mid}} \cdot x_1 = -0.2438 \qquad \frac{\partial L}{\partial \alpha_2} = \delta_{\text{mid}} \cdot x_2 = -0.1219 \qquad \frac{\partial L}{\partial b} = \delta_{\text{mid}} = -0.2438$$

- 5個の勾配が**掛け算と代入だけ**で出揃った。新しい微分は1つもしていない
- 上の緑が**往路**、下の赤が**復路** — 誤差 δ が右から左へ流れ、通り道のパラメータに勾配を配っていく

検算：機械に「微分の定義」でやらせる

- 微分の定義 $\frac{L(\theta + \varepsilon) - L(\theta)}{\varepsilon}$ で、手計算を検算できる（**数値微分**）

Python

```
1 import numpy as np
2
3 x1, x2, y = 1.0, 0.5, 1.0
4
5 def loss(a1, a2, b, w, c):
6     z = a1*x1 + a2*x2 + b
7     h = max(0.0, z)
8     u = w*h + c
9     p = 1 / (1 + np.exp(-u))
10    return -np.log(p)
11
12 params = dict(a1=0.3, a2=-0.2, b=0.1, w=0.5, c=-0.1)
13 base = loss(**params)
14 print(f"L = {base:.4f}")
15
16 eps = 1e-6
17 for name in params:
18     shifted = dict(params)
```

L = 0.6685
∂L/∂a1 = -0.2438
∂L/∂a2 = -0.1219
∂L/∂b = -0.2438
∂L/∂w = -0.1463
∂L/∂c = -0.4875

- 手計算と **完全一致**（a1: −0.2438, a2: −0.1219, b: −0.2438, w: −0.1463, c: −0.4875）
- ではなぜ実用で数値微分を使わない？ — パラメータ100万個なら loss を **100万回** 計算することになる。backprop は **1往復** で全部出る

一般形：何層でも同じ漸化式

- 層がいくつあっても、復路は **同じ形の繰り返し**：

$\delta^{(\ell)} = \delta^{(\ell+1)} \cdot w^{(\ell+1)} \cdot f'(z^{(\ell)})$

- 「次の層の δ に、重みと活性化の微分を掛けるだけ」
- 各層のパラメータの勾配は $\delta^{(\ell)} \times$ （その層への入力）

❶ 名前の由来

誤差（ δ ）が出力から入力へ**逆向きに伝播**していく — **誤差逆伝播法（backpropagation）**。仕組みの正体は連鎖律の使い回し。

復路で効いてくる設計たち

- 往路の途中の値 (z, h) を保存しておくのは復路で使うため
 - ディープラーニングがメモリを食う理由のひとつ
- $\text{ReLU}'(z)$ は **1 か 0** — 掛けても勾配が痩せない（生きている経路はそのまま通す）
 - シグモイドを中間に使うと微分が最大 0.25 — 層を重ねるたびに勾配が薄まる（**勾配消失 (vanishing gradient)**）
 - 中間層に ReLU が選ばれる大きな理由

💡 ヒント

最小単位の回の「 $p - y$ のきれいさ」も、ReLU の「1 か 0」も、 ぜんぶ**復路の掛け算を壊さないための部品選び**だった。

まとめ

- 勾配の計算は、**連鎖律で出力側から掛けて貯める** — 展開不要、微分は部品ごと
- 手計算1往復：forward の値を保存 → δ を逆流させて各勾配
- **数値微分** は検算には最高、実用には遅すぎる
- 何層でも同じ漸化式 $\delta^{(\ell)} = \delta^{(\ell+1)} w^{(\ell+1)} f'(z^{(\ell)})$ が回るだけ

📌 次回予告

部品はすべて揃った。次回、**NumPy だけで** 多層ネットワークを組み、 講義の初回に見せた「機械が自力で見つけた曲がる境界」を **自分のコードで再現** する。